Listing 20.8 Token replacement in `RouteAttributes`

```
[Route("api/[controller]")]                    ❶
public abstract class BaseController { }       ❷

public class CarController : BaseController
{
    [Route("[action]")]                        ❸
    [Route("ignition")]                        ❹
    [Route("/start-car")]                      ❺
    public IActionResult Start()
    {
         /* method implementation*/
    }
}
```

❶ You can apply attributes to a base class, and derived classes will inherit them.

❷ Token replacement happens last, so [controller] is replaced with “car” not “base”.

❸ Combines and replaces tokens to give the “api/car/start” template

❹ Combines and replaces tokens to give the “api/car/ignition” template

❺ Does not combine with base attributes because it starts with /, so it remains as “start-car”

WARNING If you use token replacement for `[controller]` or `[action]`, remember that renaming classes and methods will change your public API. If that worries you, you can stick to using static strings like `"car"` instead.

When combined with everything you learned in chapter 6, we’ve covered pretty much everything there is to know about attribute routing. There’s just one more thing to consider: handling different HTTP request types like `GET` and `POST`.

20.3.3 Handling HTTP verbs with attribute routing

In Razor Pages, the HTTP verb, such as `GET` or `POST`, isn’t part of the routing process. The `RoutingMiddleware` determines which Razor